JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class Period

java.lang.Object
    java.time.Period

All Implemented Interfaces:

`Serializable`, `ChronoPeriod`, `TemporalAmount`

```
public final class Period
extends Object
implements ChronoPeriod, Serializable
```

A date-based amount of time in the ISO-8601 calendar system, such as '2 years, 3 months and 4 days'.

This class models a quantity or amount of time in terms of years, months and days. See `Duration` for the time-based equivalent to this class.

Durations and periods differ in their treatment of daylight savings time when added to `ZonedDateTime`. A `Duration` will add an exact number of seconds, thus a duration of one day is always exactly 24 hours. By contrast, a `Period` will add a conceptual day, trying to maintain the local time.

For example, consider adding a period of one day and a duration of one day to 18:00 on the evening before a daylight savings gap. The `Period` will add the conceptual day and result in a `ZonedDateTime` at 18:00 the following day. By contrast, the `Duration` will add exactly 24 hours, resulting in a `ZonedDateTime` at 19:00 the following day (assuming a one hour DST gap).

The supported units of a period are `YEARS`, `MONTHS` and `DAYS`. All three fields are always present, but may be set to zero.

The ISO-8601 calendar system is the modern civil calendar system used today in most of the world. It is equivalent to the proleptic Gregorian calendar system, in which today's rules for leap years are applied for all time.

The period is modeled as a directed amount of time, meaning that individual parts of the period may be negative.

This is a value-based class; programmers should treat instances that are equal as interchangeable and should not use instances for synchronization, or unpredictable behavior may occur. For example, in a future release, synchronization may fail. The `equals` method should be used for comparisons.

Implementation Requirements:

This class is immutable and thread-safe.

Since:

1.8

See Also:

Serialized Form

Field Summary

Fields

Modifier and Type	Field	Description
static final Period	ZERO	A constant for a period of zero.

Method Summary

All Methods | **Static Methods**

Modifier and Type	Method	Description
Temporal	addTo (Temporal temporal)	Adds this period to the specified temporal object.
static Period	between (LocalDate startDateIncl LocalDate endDateExclusi	Obtains a Period consisting of the number of years, months, and days between two dates.
boolean	equals(Object obj)	Checks if this period is equal to another period.
static Period	from (TemporalAmount amount)	Obtains an instance of Period from a temporal amount.
long	get(TemporalUnit unit)	Gets the value of the requested unit.
IsoChronology	getChronology()	Gets the chronology of this period, which is the ISO calendar system.
int	getDays()	Gets the amount of days of this period.
int	getMonths()	Gets the amount of months of this period.
List<TemporalUnit>	getUnits()	

		Gets the set of units supported by this period.
int	**getYears**()	Gets the amount of years of this period.
int	**hashCode**()	A hash code for this period.
boolean	**isNegative**()	Checks if any of the three units of this period are negative.
boolean	**isZero**()	Checks if all three units of this period are zero.
Period	**minus**(TemporalAmount amountTo	Returns a copy of this period with the specified period subtracted.
Period	**minusDays**(long daysToSubtract)	Returns a copy of this period with the specified days subtracted.
Period	**minusMonths**(long monthsToSubtract)	Returns a copy of this period with the specified months subtracted.
Period	**minusYears**(long yearsToSubtract)	Returns a copy of this period with the specified years subtracted.
Period	**multipliedBy**(int scalar)	Returns a new instance with each element in this period multiplied by the specified scalar.
Period	**negated**()	Returns a new instance with each amount in this period negated.
Period	**normalized**()	Returns a copy of this period with the years and months normalized.
static Period	**of**(int years, int months, int days)	Obtains a Period representing a number

		of years, months and days.
static Period	**ofDays**(int days)	Obtains a Period representing a number of days.
static Period	**ofMonths**(int months)	Obtains a Period representing a number of months.
static Period	**ofWeeks**(int weeks)	Obtains a Period representing a number of weeks.
static Period	**ofYears**(int years)	Obtains a Period representing a number of years.
static Period	**parse**(CharSequence text)	Obtains a Period from a text string such as PnYnMnD.
Period	**plus**(TemporalAmount amountTo	Returns a copy of this period with the specified period added.
Period	**plusDays**(long daysToAdd)	Returns a copy of this period with the specified days added.
Period	**plusMonths**(long monthsToAdd)	Returns a copy of this period with the specified months added.
Period	**plusYears**(long yearsToAdd)	Returns a copy of this period with the specified years added.
Temporal	**subtractFrom**(Temporal temporal)	Subtracts this period from the specified temporal object.
String	**toString**()	Outputs this period as a String, such as P6Y3M1D.

long	**toTotalMonths**()	Gets the total number of months in this period.
Period	**withDays**(int days)	Returns a copy of this period with the specified amount of days.
Period	**withMonths**(int months)	Returns a copy of this period with the specified amount of months.
Period	**withYears**(int years)	Returns a copy of this period with the specified amount of years.

Methods declared in class Object

clone, finalize, getClass, notify, notifyAll, wait, wait, wait

Field Details

ZERO

public static final Period ZERO

A constant for a period of zero.

Method Details

ofYears

public static Period ofYears(int years)

Obtains a Period representing a number of years.

The resulting period will have the specified years. The months and days units will be zero.

Parameters:

years - the number of years, positive or negative

Returns:

the period of years, not null

ofMonths

public static Period ofMonths(int months)

Obtains a `Period` representing a number of months.

The resulting period will have the specified months. The years and days units will be zero.

Parameters:

`months` - the number of months, positive or negative

Returns:

the period of months, not null

ofWeeks

```
public static Period ofWeeks(int weeks)
```

Obtains a `Period` representing a number of weeks.

The resulting period will be day-based, with the amount of days equal to the number of weeks multiplied by 7. The years and months units will be zero.

Parameters:

`weeks` - the number of weeks, positive or negative

Returns:

the period, with the input weeks converted to days, not null

ofDays

```
public static Period ofDays(int days)
```

Obtains a `Period` representing a number of days.

The resulting period will have the specified days. The years and months units will be zero.

Parameters:

`days` - the number of days, positive or negative

Returns:

the period of days, not null

of

```
public static Period of(int years,
                        int months,
                        int days)
```

Obtains a `Period` representing a number of years, months and days.

This creates an instance based on years, months and days.

Parameters:

`years` - the amount of years, may be negative

`months` - the amount of months, may be negative

`days` - the amount of days, may be negative

Returns:

the period of years, months and days, not null

from

```
public static Period from(TemporalAmount amount)
```

Obtains an instance of `Period` from a temporal amount.

This obtains a period based on the specified amount. A `TemporalAmount` represents an amount of time, which may be date-based or time-based, which this factory extracts to a `Period`.

The conversion loops around the set of units from the amount and uses the `YEARS`, `MONTHS` and `DAYS` units to create a period. If any other units are found then an exception is thrown.

If the amount is a `ChronoPeriod` then it must use the ISO chronology.

Parameters:

`amount` - the temporal amount to convert, not null

Returns:

the equivalent period, not null

Throws:

`DateTimeException` - if unable to convert to a `Period`

`ArithmeticException` - if the amount of years, months or days exceeds an int

parse

```
public static Period parse(CharSequence text)
```

Obtains a `Period` from a text string such as `PnYnMnD`.

This will parse the string produced by `toString()` which is based on the ISO-8601 period formats `PnYnMnD` and `PnW`.

The string starts with an optional sign, denoted by the ASCII negative or positive symbol. If negative, the whole period is negated. The ASCII letter "P" is next in upper or lower case. There are then four sections, each consisting of a number and a suffix. At least one of the four sections must be present. The

sections have suffixes in ASCII of "Y", "M", "W" and "D" for years, months, weeks and days, accepted in upper or lower case. The suffixes must occur in order. The number part of each section must consist of ASCII digits. The number may be prefixed by the ASCII negative or positive symbol. The number must parse to an `int`.

The leading plus/minus sign, and negative values for other units are not part of the ISO-8601 standard. In addition, ISO-8601 does not permit mixing between the `PnYnMnD` and `PnW` formats. Any week-based input is multiplied by 7 and treated as a number of days.

For example, the following are valid inputs:

```
"P2Y"             -- Period.ofYears(2)
"P3M"             -- Period.ofMonths(3)
"P4W"             -- Period.ofWeeks(4)
"P5D"             -- Period.ofDays(5)
"P1Y2M3D"         -- Period.of(1, 2, 3)
"P1Y2M3W4D"       -- Period.of(1, 2, 25)
"P-1Y2M"          -- Period.of(-1, 2, 0)
"-P1Y2M"          -- Period.of(-1, -2, 0)
```

Parameters:

`text` - the text to parse, not null

Returns:

the parsed period, not null

Throws:

`DateTimeParseException` - if the text cannot be parsed to a period

between

```
public static Period between(LocalDate startDateInclusive,
                             LocalDate endDateExclusive)
```

Obtains a `Period` consisting of the number of years, months, and days between two dates.

The start date is included, but the end date is not. The period is calculated by removing complete months, then calculating the remaining number of days, adjusting to ensure that both have the same sign. The number of months is then split into years and months based on a 12 month year. A month is considered if the end day-of-month is greater than or equal to the start day-of-month. For example, from `2010-01-15` to `2011-03-18` is one year, two months and three days.

The result of this method can be a negative period if the end is before the start. The negative sign will be the same in each of year, month and day.

Parameters:

`startDateInclusive` - the start date, inclusive, not null

`endDateExclusive` - the end date, exclusive, not null

Returns:

the period between this date and the end date, not null

See Also:

`ChronoLocalDate.until(ChronoLocalDate)`

get

```
public long get(TemporalUnit unit)
```

Gets the value of the requested unit.

This returns a value for each of the three supported units, `YEARS`, `MONTHS` and `DAYS`. All other units throw an exception.

Specified by:

`get` in interface `ChronoPeriod`

Specified by:

`get` in interface `TemporalAmount`

Parameters:

`unit` - the `TemporalUnit` for which to return the value

Returns:

the long value of the unit

Throws:

`DateTimeException` - if the unit is not supported

`UnsupportedTemporalTypeException` - if the unit is not supported

getUnits

```
public List<TemporalUnit> getUnits()
```

Gets the set of units supported by this period.

The supported units are `YEARS`, `MONTHS` and `DAYS`. They are returned in the order years, months, days.

This set can be used in conjunction with `get(TemporalUnit)` to access the entire state of the period.

Specified by:

`getUnits` in interface `ChronoPeriod`

Specified by:

`getUnits` in interface `TemporalAmount`

Returns:

a list containing the years, months and days units, not null

getChronology

```
public IsoChronology getChronology()
```

Gets the chronology of this period, which is the ISO calendar system.

The `Chronology` represents the calendar system in use. The ISO-8601 calendar system is the modern civil calendar system used today in most of the world. It is equivalent to the proleptic Gregorian calendar system, in which today's rules for leap years are applied for all time.

Specified by:

`getChronology` in interface `ChronoPeriod`

Returns:

the ISO chronology, not null

isZero

```
public boolean isZero()
```

Checks if all three units of this period are zero.

A zero period has the value zero for the years, months and days units.

Specified by:

`isZero` in interface `ChronoPeriod`

Returns:

true if this period is zero-length

isNegative

```
public boolean isNegative()
```

Checks if any of the three units of this period are negative.

This checks whether the years, months or days units are less than zero.

Specified by:

`isNegative` in interface `ChronoPeriod`

Returns:

true if any unit of this period is negative

getYears

```
public int getYears()
```

Gets the amount of years of this period.

This returns the years unit.

The months unit is not automatically normalized with the years unit. This means that a period of "15 months" is different to a period of "1 year and 3 months".

Returns:

the amount of years of this period, may be negative

getMonths

```
public int getMonths()
```

Gets the amount of months of this period.

This returns the months unit.

The months unit is not automatically normalized with the years unit. This means that a period of "15 months" is different to a period of "1 year and 3 months".

Returns:

the amount of months of this period, may be negative

getDays

```
public int getDays()
```

Gets the amount of days of this period.

This returns the days unit.

Returns:

the amount of days of this period, may be negative

withYears

```
public Period withYears(int years)
```

Returns a copy of this period with the specified amount of years.

This sets the amount of the years unit in a copy of this period. The months and days units are unaffected.

The months unit is not automatically normalized with the years unit. This means that a period of "15 months" is different to a period of "1 year and 3 months".

This instance is immutable and unaffected by this method call.

Parameters:

`years` - the years to represent, may be negative

Returns:

a `Period` based on this period with the requested years, not null

withMonths

```
public Period withMonths(int months)
```

Returns a copy of this period with the specified amount of months.

This sets the amount of the months unit in a copy of this period. The years and days units are unaffected.

The months unit is not automatically normalized with the years unit. This means that a period of "15 months" is different to a period of "1 year and 3 months".

This instance is immutable and unaffected by this method call.

Parameters:

`months` - the months to represent, may be negative

Returns:

a `Period` based on this period with the requested months, not null

withDays

```
public Period withDays(int days)
```

Returns a copy of this period with the specified amount of days.

This sets the amount of the days unit in a copy of this period. The years and months units are unaffected.

This instance is immutable and unaffected by this method call.

Parameters:

`days` - the days to represent, may be negative

Returns:

a Period based on this period with the requested days, not null

plus

```
public Period plus(TemporalAmount amountToAdd)
```

Returns a copy of this period with the specified period added.

This operates separately on the years, months and days. No normalization is performed.

For example, "1 year, 6 months and 3 days" plus "2 years, 2 months and 2 days" returns "3 years, 8 months and 5 days".

The specified amount is typically an instance of Period. Other types are interpreted using from(TemporalAmount).

This instance is immutable and unaffected by this method call.

Specified by:

plus in interface ChronoPeriod

Parameters:

amountToAdd - the amount to add, not null

Returns:

a Period based on this period with the requested period added, not null

Throws:

DateTimeException - if the specified amount has a non-ISO chronology or contains an invalid unit

ArithmeticException - if numeric overflow occurs

plusYears

```
public Period plusYears(long yearsToAdd)
```

Returns a copy of this period with the specified years added.

This adds the amount to the years unit in a copy of this period. The months and days units are unaffected. For example, "1 year, 6 months and 3 days" plus 2 years returns "3 years, 6 months and 3 days".

This instance is immutable and unaffected by this method call.

Parameters:

yearsToAdd - the years to add, positive or negative

Returns:

a Period based on this period with the specified years added, not null

Throws:

`ArithmeticException` - if numeric overflow occurs

plusMonths

```
public Period plusMonths(long monthsToAdd)
```

Returns a copy of this period with the specified months added.

This adds the amount to the months unit in a copy of this period. The years and days units are unaffected. For example, "1 year, 6 months and 3 days" plus 2 months returns "1 year, 8 months and 3 days".

This instance is immutable and unaffected by this method call.

Parameters:

`monthsToAdd` - the months to add, positive or negative

Returns:

a `Period` based on this period with the specified months added, not null

Throws:

`ArithmeticException` - if numeric overflow occurs

plusDays

```
public Period plusDays(long daysToAdd)
```

Returns a copy of this period with the specified days added.

This adds the amount to the days unit in a copy of this period. The years and months units are unaffected. For example, "1 year, 6 months and 3 days" plus 2 days returns "1 year, 6 months and 5 days".

This instance is immutable and unaffected by this method call.

Parameters:

`daysToAdd` - the days to add, positive or negative

Returns:

a `Period` based on this period with the specified days added, not null

Throws:

`ArithmeticException` - if numeric overflow occurs

minus

```
public Period minus(TemporalAmount amountToSubtract)
```

Returns a copy of this period with the specified period subtracted.

This operates separately on the years, months and days. No normalization is performed.

For example, "1 year, 6 months and 3 days" minus "2 years, 2 months and 2 days" returns "-1 years, 4 months and 1 day".

The specified amount is typically an instance of `Period`. Other types are interpreted using `from(TemporalAmount)`.

This instance is immutable and unaffected by this method call.

Specified by:

`minus` in interface `ChronoPeriod`

Parameters:

`amountToSubtract` - the amount to subtract, not null

Returns:

a `Period` based on this period with the requested period subtracted, not null

Throws:

`DateTimeException` - if the specified amount has a non-ISO chronology or contains an invalid unit

`ArithmeticException` - if numeric overflow occurs

minusYears

```
public Period minusYears(long yearsToSubtract)
```

Returns a copy of this period with the specified years subtracted.

This subtracts the amount from the years unit in a copy of this period. The months and days units are unaffected. For example, "1 year, 6 months and 3 days" minus 2 years returns "-1 years, 6 months and 3 days".

This instance is immutable and unaffected by this method call.

Parameters:

`yearsToSubtract` - the years to subtract, positive or negative

Returns:

a `Period` based on this period with the specified years subtracted, not null

Throws:

`ArithmeticException` - if numeric overflow occurs

minusMonths

```
public Period minusMonths(long monthsToSubtract)
```

Returns a copy of this period with the specified months subtracted.

This subtracts the amount from the months unit in a copy of this period. The years and days units are unaffected. For example, "1 year, 6 months and 3 days" minus 2 months returns "1 year, 4 months and 3 days".

This instance is immutable and unaffected by this method call.

Parameters:

`monthsToSubtract` - the years to subtract, positive or negative

Returns:

a `Period` based on this period with the specified months subtracted, not null

Throws:

`ArithmeticException` - if numeric overflow occurs

minusDays

```
public Period minusDays(long daysToSubtract)
```

Returns a copy of this period with the specified days subtracted.

This subtracts the amount from the days unit in a copy of this period. The years and months units are unaffected. For example, "1 year, 6 months and 3 days" minus 2 days returns "1 year, 6 months and 1 day".

This instance is immutable and unaffected by this method call.

Parameters:

`daysToSubtract` - the days to subtract, positive or negative

Returns:

a `Period` based on this period with the specified days subtracted, not null

Throws:

`ArithmeticException` - if numeric overflow occurs

multipliedBy

```
public Period multipliedBy(int scalar)
```

Returns a new instance with each element in this period multiplied by the specified scalar.

This returns a period with each of the years, months and days units individually multiplied. For example, a period of "2 years, -3 months and 4 days" multiplied by 3 will return "6 years, -9 months and 12 days". No normalization is performed.

Specified by:

`multipliedBy` in interface `ChronoPeriod`

Parameters:

`scalar` - the scalar to multiply by, not null

Returns:

a `Period` based on this period with the amounts multiplied by the scalar, not null

Throws:

`ArithmeticException` - if numeric overflow occurs

negated

```
public Period negated()
```

Returns a new instance with each amount in this period negated.

This returns a period with each of the years, months and days units individually negated. For example, a period of "2 years, -3 months and 4 days" will be negated to "-2 years, 3 months and -4 days". No normalization is performed.

Specified by:

`negated` in interface `ChronoPeriod`

Returns:

a `Period` based on this period with the amounts negated, not null

Throws:

`ArithmeticException` - if numeric overflow occurs, which only happens if one of the units has the value `Integer.MIN_VALUE`

normalized

```
public Period normalized()
```

Returns a copy of this period with the years and months normalized.

This normalizes the years and months units, leaving the days unit unchanged. The months unit is adjusted to have an absolute value less than 12, with the years unit being adjusted to compensate. For example, a period of "1 Year and 15 months" will be normalized to "2 years and 3 months".

The sign of the years and months units will be the same after normalization. For example, a period of "1 year and -25 months" will be normalized to "-1 year and -1 month".

This instance is immutable and unaffected by this method call.

Specified by:

`normalized` in interface `ChronoPeriod`

Returns:

a `Period` based on this period with excess months normalized to years, not null

Throws:

`ArithmeticException` - if numeric overflow occurs

toTotalMonths

```
public long toTotalMonths()
```

Gets the total number of months in this period.

This returns the total number of months in the period by multiplying the number of years by 12 and adding the number of months.

This instance is immutable and unaffected by this method call.

Returns:

the total number of months in the period, may be negative

addTo

```
public Temporal addTo(Temporal temporal)
```

Adds this period to the specified temporal object.

This returns a temporal object of the same observable type as the input with this period added. If the temporal has a chronology, it must be the ISO chronology.

In most cases, it is clearer to reverse the calling pattern by using `Temporal.plus(TemporalAmount)`.

```
// these two lines are equivalent, but the second approach is recommended
dateTime = thisPeriod.addTo(dateTime);
dateTime = dateTime.plus(thisPeriod);
```

The calculation operates as follows. First, the chronology of the temporal is checked to ensure it is ISO chronology or null. Second, if the months are zero, the years are added if non-zero, otherwise the combination of years and months is added if non-zero. Finally, any days are added.

This approach ensures that a partial period can be added to a partial date. For example, a period of years and/or months can be added to a `YearMonth`, but a

period including days cannot. The approach also adds years and months together when necessary, which ensures correct behaviour at the end of the month.

This instance is immutable and unaffected by this method call.

Specified by:

`addTo` in interface `ChronoPeriod`

Specified by:

`addTo` in interface `TemporalAmount`

Parameters:

`temporal` - the temporal object to adjust, not null

Returns:

an object of the same type with the adjustment made, not null

Throws:

`DateTimeException` - if unable to add

`ArithmeticException` - if numeric overflow occurs

subtractFrom

`public Temporal subtractFrom(Temporal temporal)`

Subtracts this period from the specified temporal object.

This returns a temporal object of the same observable type as the input with this period subtracted. If the temporal has a chronology, it must be the ISO chronology.

In most cases, it is clearer to reverse the calling pattern by using `Temporal.minus(TemporalAmount)`.

```
// these two lines are equivalent, but the second approach is recommended
dateTime = thisPeriod.subtractFrom(dateTime);
dateTime = dateTime.minus(thisPeriod);
```

The calculation operates as follows. First, the chronology of the temporal is checked to ensure it is ISO chronology or null. Second, if the months are zero, the years are subtracted if non-zero, otherwise the combination of years and months is subtracted if non-zero. Finally, any days are subtracted.

This approach ensures that a partial period can be subtracted from a partial date. For example, a period of years and/or months can be subtracted from a `YearMonth`, but a period including days cannot. The approach also subtracts

years and months together when necessary, which ensures correct behaviour at the end of the month.

This instance is immutable and unaffected by this method call.

Specified by:

`subtractFrom` in interface `ChronoPeriod`

Specified by:

`subtractFrom` in interface `TemporalAmount`

Parameters:

`temporal` - the temporal object to adjust, not null

Returns:

an object of the same type with the adjustment made, not null

Throws:

`DateTimeException` - if unable to subtract

`ArithmeticException` - if numeric overflow occurs

equals

```
public boolean equals(Object obj)
```

Checks if this period is equal to another period.

The comparison is based on the type `Period` and each of the three amounts. To be equal, the years, months and days units must be individually equal. Note that this means that a period of "15 Months" is not equal to a period of "1 Year and 3 Months".

Specified by:

`equals` in interface `ChronoPeriod`

Overrides:

`equals` in class `Object`

Parameters:

`obj` - the object to check, null returns false

Returns:

true if this is equal to the other period

See Also:

`Object.hashCode()`, `HashMap`

hashCode

```
public int hashCode()
```

A hash code for this period.

Specified by:

`hashCode` in interface `ChronoPeriod`

Overrides:

`hashCode` in class `Object`

Returns:

a suitable hash code

See Also:

`Object.equals(java.lang.Object)`,
`System.identityHashCode(Object)`

toString

```
public String toString()
```

Outputs this period as a `String`, such as `P6Y3M1D`.

The output will be in the ISO-8601 period format. A zero period will be represented as zero days, 'P0D'.

Specified by:

`toString` in interface `ChronoPeriod`

Overrides:

`toString` in class `Object`

Returns:

a string representation of this period, not null

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.